

Intelligent Offloading Balance for Vehicular Edge Computing and Networks

Yu Wu¹, Xuming Fang¹, Senior Member, IEEE, Geyong Min², Member, IEEE, Hongyang Chen¹, Senior Member, IEEE, and Chunbo Luo¹, Senior Member, IEEE

Abstract—With the explosion of connected devices and Internet-of-Things (IoT) services in the smart city, the challenge to meet the demands of urban computing is increasingly prominent. Recent advances in vehicle-to-everything (V2X) communications promote urban Internet-connected vehicles to become excellent candidates for computing tasks. However, due to the limited computing capacity of vehicles, conducting computation in the vehicular networks themselves is insufficient to satisfy the demands of smart city applications. Edge computing, which delivers computing tasks to edge servers (e.g., base stations, BSs, or roadside units, RSUs) with plenty of computing resources, could be a possible solution. However, the static deployment of edge servers may cause severe load unbalance among servers in both real-time communication and computation, thereby decreasing the system performance. This paper explores a two-hop vehicle-assisted edge computing network framework in which vehicles are able to offload the tasks beyond their capabilities to underloaded edge servers relaying via neighbor vehicles. According to the state of the time-varying vehicular environment and the dynamic traffic loads among RSUs, we formulate the task offloading, relay node selection, and resources allocations problem as a Markov decision process (MDP) aiming at maximizing the performance of the computation offloading capacity with the considerations of load balancing and latency constraints. We propose a deep reinforcement learning (DRL) algorithm with a DNN as Q action-value function approximator to solve this problem. Extensive simulation results reveal that the proposed scheme can significantly improve the system performance compared to other state-of-the-art algorithms.

Index Terms—Computation offloading, load balance, vehicular edge computing, deep reinforcement learning.

Received 24 November 2022; revised 12 November 2023, 4 March 2024, and 22 December 2024; accepted 7 February 2025. Date of publication 21 March 2025; date of current version 5 May 2025. This work was supported in part by NSFC under Grant 62071393, in part by the NSFC High-Speed Rail Joint Foundation under Grant U1834210, in part by Sichuan Provincial Applied Basic Research Project under Grant 2020YJ0218, in part by the Fundamental Research Funds for the Central Universities under Grant 2682023ZTPY058, and in part by China Scholarship Council under Grant 201907000092. The work of Hongyang Chen was supported in part by the National Natural Science Foundation of China under Grant 62271452. The Associate Editor for this article was K. P. Tran. (Corresponding author: Xuming Fang.)

Yu Wu and Xuming Fang are with the Key Laboratory of Information Coding and Transmission, Southwest Jiaotong University, Chengdu 610032, China (e-mail: linda_swjtu@126.com; xmfang@swjtu.edu.cn).

Geyong Min and Chunbo Luo are with the College of Engineering, Mathematics, and Physical Sciences, University of Exeter, EX4 4PY Exeter, U.K. (e-mail: g.min@exeter.ac.uk; C.Luo@exeter.ac.uk).

Hongyang Chen is with the Research Center for Intelligent Network, Zhejiang Laboratory, Hangzhou, Zhejiang 311121, China (e-mail: dr.h.chen@ieee.org).

Digital Object Identifier 10.1109/TITS.2025.3549493

I. INTRODUCTION

RECENTLY, the ever increasing smart city applications (e.g., automatic driving, criminal discovery, meteorological monitoring) have led to explosive growth in the demands of urban computing. How to transmit and compute the massive Internet-of-Things (IoT) data for knowledge discovery of smart city is quite challenging [1]. With the development of 5G/B5G communications and Internet-of-Vehicles (IoV) technologies, the urban Internet-connected vehicles, which are equipped with onboard units to perform data communication, computation, and caching, have been considered as ideal candidates for collecting and executing the computing tasks [2]. However, it is not guaranteed to provide high-quality services, resulting from the limited onboard computing capabilities of vehicles.

Cloud offloading and edge offloading are currently two possible solutions to obtain plenty of computing resources. However, cloud computing, which offloads such massive collected computing data to remote and centralized cloud servers, may lead to high service delay, transmission energy consumption, and backhaul congestions [3]. Mobile edge computing (MEC) offloads the computing data by 1-hop transmission to the associated edge server (e.g., BSs, or RSUs) with relatively powerful computing and caching capabilities, which can provide high-bandwidth and low-latency computing services in close proximity to vehicles. Nevertheless, the fixed and nonuniform radio coverage of statically deployed edge servers, daily human activities, and irregular layouts among living areas and working areas cause spatial-temporal dynamic traffic loads, thereby resulting in network congestion and load unbalance problems [4], [5], [6], [7]. Driven by the above issues, it is necessary to study a vehicle-collaborative computation offloading scheme to achieve scalable task offloading and processing as well as traffic load balance.

There are researches on vehicular edge computing networks including the network framework, offloading policies, and related resource allocations. Yang et al. [2] proposed a contract-based incentive mechanism to motivate vehicles on the freeway to form a vehicle platoon, in which vehicles' computing resources are centrally controlled to assist BS's latency-sensitive computing tasks. Liu et al. [4] proposed a vehicle-assisted task offloading scheme in which available vehicles serve as mobile computing servers to extend the computing service range of fixed MEC servers. They formulated the computation offloading and resource allocation problem as a semi-Markov process and the DRL method

was applied to find the optimal policy to maximize the network economic utility. Fan et al. [6] considered buses or commuting vehicles with regular mobility pattern to assist traffic offloading. They formulated the offloading association problem to maximum the load-balanced offloading throughput of APs and relay vehicles. Dai et al. [7] proposed a load-balanced computation offloading scheme for a multivehicle multiserver vehicular edge computing network. The partial offloading scheme jointly considered RSU selection, load balancing, and computing resource allocation to minimize the load-balance weighted total task latency. Specifically, the total delay is the maximum value of the locally executed part and the RSU executed part, and the RSU executed part includes a vehicle's moving time to the selected RSU. Qiao et al. [9] proposed a 3-layered hierarchical framework and collaborative task offloading for mobile vehicular edge multi-access networks. Based on a two-side matching scheme, computing tasks can be offloaded to the vehicular cloudlets or MEC platforms to achieve minimal vehicular utility of service latency and network utility of load balancing. For delay-tolerant data traffic, Li et al. [10] jointly optimized the transmission decision, caching option, and computing execution place to minimize the system cost of network overhead and computing latency.

However, the existing researches such as [2] and [4] mainly consider vehicles as computing service providers, whereas these computing resource-constrained vehicles mostly are busy enough with their own computing tasks (e.g., self-generated IoV computing tasks or collected smart-city computing tasks). In this case, they can still act as relay nodes of task offloading if they have available communication resources. Hence, it is important to explore the multi-dimensional resources of vehicular networks to achieve multi-dimensional diversity gain. In addition, very few works such as [6], [7], and [9] studied the load balance. They either did not consider the resource allocation and computing latency [6], or only studied the load-balanced latency for latency-tolerant tasks [7], or the load balance is balancing computing and communication resources rather than balancing traffic load among multiple servers.

Hence, in this paper, we consider Internet-connected vehicles driving in a smart city generating computing tasks of IoV services or collecting computing tasks from smart city applications. For those computing tasks (either latency-constrained or latency-tolerant tasks) beyond their onboard computing capabilities, vehicles choose to offload them to underloaded RSUs via vehicle-to-RSU (V2R) communications or vehicle-to-vehicle (V2V) communications. We propose a DRL-based computation offloading policy to maximize the computation offloading capacity with considerations of the load balance and latency constraints. The contributions of this work are summarized as follows:

- 1) We propose a vehicular edge computing network framework in which vehicles can provide real-time local computing services and offload computing tasks beyond their computing capabilities to edge servers. The offloading process jointly considers vehicles' mobility, server selection, communication and computing resources allocations, and load balance.
- 2) We construct an analytical model for the vehicular edge computing networks and formulate the offloading and resource allocation optimization problem to maximize the system utility of both load-balanced offloading capacity and latency reduction.
- 3) Due to the highly dynamic topology of vehicular networks and the time-varying traffic loads in both communication and computation, we propose a DRL-based computation offloading algorithm with a deep neural network (DNN) approximator to solve the complex computation offloading optimization problem aiming at achieving the optimal long-term performance.

The rest of this paper is organized as follows. In Section II, we review the related work. In Section III, we describe the system model and formulate the problem. In Section IV, we propose the DRL-based computation offloading algorithm. In Section V, we conduct simulation experiments for performance evaluation. Finally, we conclude this paper in Section VI.

II. RELATED WORK

The computation offloading and corresponding resource allocations are of great challenges since multiple factors such as the task characteristics, dynamic network status, implicit coupling of time-varying multi-dimensional resources, and various performance requirements need to be jointly considered [30]. Peng and Shen investigated multi-dimensional resource management in UAV-assisted edge computing networks, in which UAVs and MEC servers cooperated to make association decisions and resource allocation [11]. Hou et al. investigated the reliable computation offloading scheme for latency-sensitive services, in which mobile/fix edge computing nodes can cooperate with each other by partial offloading and reliable task allocation [13]. Wang et al. proposed a collaborative vehicular edge computing framework, which allowed both horizontal and vertical collaborative computing [14]. Tang et al. studied the network slicing-based online computation offloading scheme to minimize energy consumption and service response time. DRL algorithm was applied for slice selection and KKT conditions and bisection method were used to achieve resource allocation [16]. Feng et al. formulated a power consumption optimization problem with guaranteed network stability for vehicular edge networks, in which non-cooperative game and bipartite graph was used for channel allocation and Lyapunov optimization method were used for computation resource allocation [18]. Wang et al. studied a multiuser non-cooperative game to adjust the offloading ratio in vehicular edge networks, in which the distance between vehicles and the edge server, communication model and multi-vehicle competition were considered in the construction of utility function [19]. Ning et al. proposed a partial computation offloading and adaptive task scheduling algorithm to maximize the overall system-wide profit. A two-sided matching algorithm was applied to derive the optimal transmission scheduling discipline and convex optimization was used to obtain the offloading ratio, and finally a non-cooperative game is constructed to derive vehicles profit [20].

Zhang et al. proposed a DRL-based MEC offloading scheme to determine the optimal target server and offloading mode, and a redundant offloading algorithm was presented to improve the task offloading reliability [22]. Zhan et al. jointly considered task characteristics, dynamic wireless environment, and vehicle movements in a computation offloading scheduling problem for vehicular edge computing to tradeoff between task latency and energy consumption [24]. However, the above work, such as [10] and [22] either did not consider computing or communication resource allocation or the optimization objective is the network economic utility.

In summary, existing studies differs from each other on different MEC-based/enabled scenarios (e.g., IoT networks [3], [8], [31], Vehicular networks [2], [4], [7], [9], [10], and Unmanned Aerial Vehicle [11], [12]) and technologies (e.g., Software Defined Networks/Network Function Virtualized [6], [10], [13], [14] and Network slicing [15], [16]). These works also vary with different optimization objectives (e.g., latency and energy efficiency), constraints (e.g., mobility, deadline, energy, storage, computing power, and fronthaul/backhaul capacities), and solving methods and algorithms (e.g., game theory [17], [18], [19], matching theory [17], convex optimization [20], lyapunov optimization theory [17], and artificial intelligence (AI) [1], [4], [6], [8], [11], [15], [21], [22]). Among these optimization methods and algorithms, AI, especially deep reinforcement learning (DRL) has its unique advantages in addressing edge computing optimization problems. Other traditional methods may suffer from uncertain input factors, dynamic conditions, and one-shot optimization [23], [32]. Whereas DRL can consider the effect of current decisions on long-term performance goals, and the optimal decisions are learned by interacting with the dynamic environment without the need of explicit system dynamics model [24], [33].

III. SYSTEM MODEL AND PROBLEM FORMULATION

A. System Architecture

We divide the system observation period W_o into N time slots t . In each time slot, the system status is regarded as constant. Consider an IoV network as illustrated in Fig. 1, a city space $\mathcal{G}$ can be discretized into n disjoint hexagonal grids with equal size [34],

$$\mathcal{G} = \{g_1, g_2, g_3 \cdots, g_n \mid g_i \cap g_j = \emptyset\}, \quad (1)$$

We consider a set of RSUs deployed in the above disjoint hexagonal grids, represented by

$$\mathcal{R} = \{r_1, r_2, r_3 \cdots, r_n\}. \quad (2)$$

RSUs are equipped with both communication and computing capabilities and r_i indicates RSU's location of g_i . In an observation period, we assume that there is a set of s vehicles associated with these RSUs:

$$\mathcal{V} = \{v_1, v_2, v_3 \cdots, v_s\} \quad (3)$$

These vehicles are equipped with onboard units to perform data communication, computation, and caching. Vehicles generate computing tasks of IoV services or collect computing tasks from smart city applications [22]. Due to limited onboard

resources, we consider vehicles, which are not able to execute all of the tasks themselves and need to offload some of the computing tasks to the RSUs, as computation offloading vehicles (COVs). Vehicles without offloading tasks in the current observation period as relay vehicles (RVs). It should be noticed that the set of COVs and RVs may vary in different observation periods.

The RSUs are connected to a centralized cloud center (CCC) by backhaul links. The CCC collects the global information, e.g., RSUs' traffic loads, available communication and computing resources, vehicles' positions and mobilities, and computation offloading requests. It can be leveraged as a central controller for signaling such as time synchronization, and a learning agent for making the optimal offloading decisions and corresponding communication and computing resources allocations.

B. Task Model and Offloading Option

At the beginning of each time slot, we assume that each vehicle v_i will collect and send its information $\mathbf{V}_i^{info} = (y_s^i, y_o^i, y_p^i, y_m^i, y_r^i, \mathbf{y}_n^i)$ to the associated RSU, where y_s^i is the information of its vehicle ID, y_o^i denotes whether it has task offloading request, $y_p^i = (x_i, y_i)$ is its latitude and longitude coordinates, $y_m^i = (s_i, \theta_i)$ is the speed vector including the velocity and heading direction, y_r^i is its associated RSU's index, $\mathbf{y}_n^i$ is the set of its neighbor vehicles' IDs within its communication range. We set the size of $\mathbf{y}_n^i$ to be a constant y_N in each slot. The above information can be obtained by vehicular sensors and via V2R/V2V communications [22]. The RSUs will upload these messages to the CCC and the CCC will have all the vehicles' information as $\mathcal{V}^{info} = \{\mathbf{V}_1^{info}, \mathbf{V}_2^{info}, \dots, \mathbf{V}_i^{info}, \dots, \mathbf{V}_s^{info}\}$. We assume that each vehicle v_i has up to one offloading request in each time slot, and it is represented by a tuple $\mathbf{Task}_i = (z_r^i, z_s^i, z_d^i)$, where z_r^i is the remaining time in ms to meet the latency constraint, z_s^i is the task's size in bit , z_d^i is the required CPU cycles to compute the task. Particularly, if the vehicle is RV (i.e., $y_o^i = 0$), the z_r^i , z_s^i , and z_d^i are all zero. Then, the task model of all vehicles is denoted by the set $\mathcal{TASK} = \{\mathbf{Task}_1, \mathbf{Task}_2, \dots, \mathbf{Task}_i, \dots, \mathbf{Task}_s\}$.

As shown in Fig. 1, in each time slot, for vehicle v_i , which can be COV or RV depending on the value of y_o^i , CCC makes the offloading decision from three task offloading options.

- 1 One-hop Local Offloading (1-hop LO): This option is only available for COVs. Directly offload the task to COV's associated RSU, if both the RSU's data transmission traffic (DTT) load and computing task (CT) load are underloaded.
- 2 Two-hops Vehicle-assisted Offloading (2-hops VAO): This option is only available for COVs. If COV's associated RSU is overloaded either in its DTT or in its CT, COV searches for available neighbor RV, whose associated RSU is underloaded in both DTT and CT, to relay its task.
- 3 Move and Wait for Future Offloading (MWFO): This option is available for RV or COV. If the vehicle is RV, then it does not have offloading request and keeps moving. If it is COV, and the COV's associated RSU is

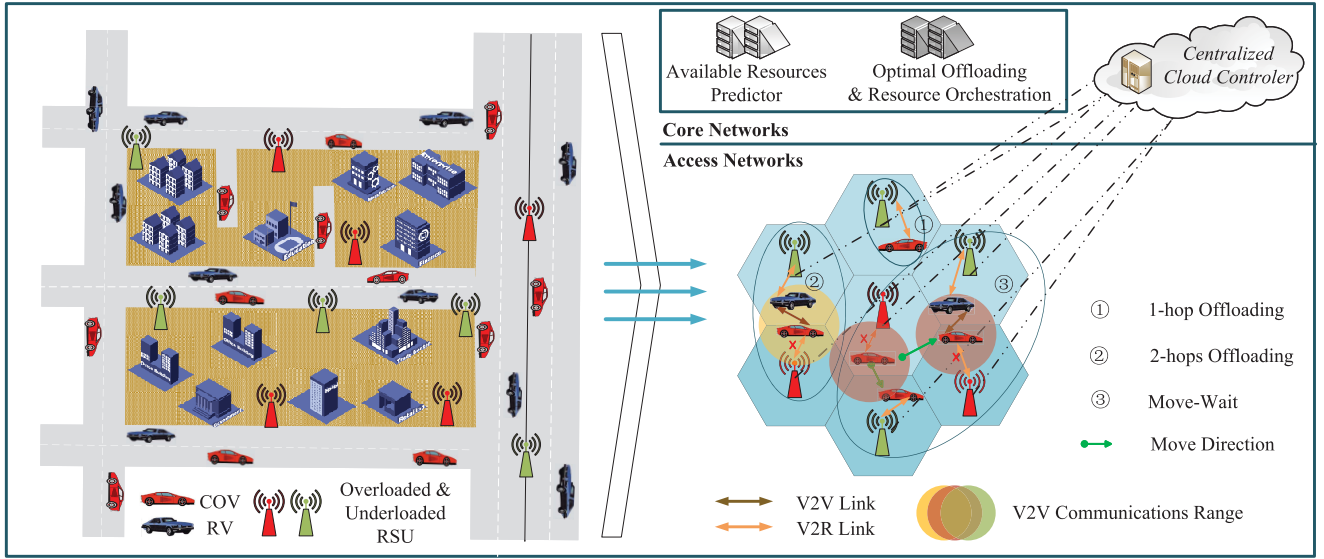


Fig. 1. Vehicular edge computing framework.

overloaded either in its DTT or in its CT, and there is no available neighbor RVs for this COV. The COV keeps moving and waits for future offloading chances till the time exceeds the latency constraint of its task.

The offloading option for $\mathbf{Task}_i$ is controlled by two binary variables, $I_{i,j}$ and $I_{i,j}^a$. $I_{i,j} = 1$ represents the $\mathbf{Task}_i$ of COV v_i is offloaded to RSU r_j (i.e., option 1), and 0 otherwise. $I_{i,j}^a = 1$ represents the $\mathbf{Task}_i$ of COV v_i is offloaded to RSU r_j via RV v_a (i.e., option 2), and 0 otherwise.

C. Offloading Latency Model

According to the offloading decisions, RSUs allocate communication and computing resources for V2R/V2V communications and task computations, respectively. V2R communications adopt low-frequency omni-directional communications and V2V communication adopt high-frequency mmWave directional communications. The spectrum allocations among RSUs are independent and orthogonal and each RSU has a total communication resource of W_l (in Hz) for V2R communications. We assume that W_l is divided into $N_{CM} \in \mathbb{N}_+$ communication resource blocks (CMRBs) with the bandwidth of W_b (in Hz), and the V2R uplink communications share these CMRBs using the technology of Orthogonal Frequency-Division Multiple Access (OFDMA). Whereas the V2V communications compete for the channel bandwidth of W_h (in Hz) by Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA). Each RSU has a total computing resource of F (in CPU cycles per second), which is further divided into $N_{CP} \in \mathbb{N}_+$ computing resource blocks (CPRBs) with the size of F_b (in CPU cycles per second) [25], [26]. The spatial-temporal traffic tide phenomenon may cause a high unbalance of DTT load and CT load among RSUs, and their available communication and computing resources are highly dynamic. In each time slot, the available communication resources and computing resources of RSU r_j are represented by $N_{CM,a}^j \leq N_{CM}$ and $N_{CP,a}^j \leq N_{CP}$, respectively. The information of RSU r_j is denoted as

$\mathbf{RSU}_j^{info} = (N_{CM,a}^j, N_{CP,a}^j)$. The CCC collects all the RSUs' information $\mathcal{RSU}^{info} = \{\mathbf{RSU}_1^{info}, \mathbf{RSU}_2^{info}, \dots, \mathbf{RSU}_n^{info}\}$.

RSU r_j allocates $N_{CM}^{i,j}$ CMRBs to offload $\mathbf{Task}_i$ by the V2R communication. Then, the effective V2R data rate for transmitting the offloading task can be expressed as R_{rv}^i ,

$$R_{rv}^i = N_{CM}^{i,j} W_b \log_2 (1 + SNR_{rv}^{i,j}) \quad (4)$$

$$SNR_{rv}^{i,j} = \frac{P_{rv}^{i,j}}{L_{rv}^{i,j} P_N} \quad (5)$$

where $P_{rv}^{i,j}$ is the transmission power in mW, P_N is the thermal noise power in mW, and

$$P_N [mW] = N_{CM}^{i,j} W_b [Hz] * 10^{\frac{N_0 [dBm/Hz]}{10}} \quad (6)$$

where N_0 is the background noise power spectrum density, as a standard temperature of 17 °C, and the thermal noise level is $N_0 = -174 [dBm/Hz]$ [27]. $L_{rv}^{i,j} [dB]$ is the path loss and can be obtained by

$$L_{rv}^{i,j} [dB] = 20 \log_{10}(d) + 20 \log_{10}(f) + 32.4 \quad (7)$$

where d is the distance between the receiver and transmitter in km, and f is the frequency band in MHz [28].

Then, similarly, the effective V2V data rate for relaying $\mathbf{Task}_i$ in option 2 can be calculated as

$$R_{vv'}^i = W_h \log_2 (1 + SINR_{vv'}^{i,a}) \quad (8)$$

The SINR of the V2V communication between COV v_i and RV v_a will be

$$SINR_{vv'}^{i,a} = \frac{\frac{P_{vv'}^{i,a} G_{i,a}^t G_{i,a}^r}{L_{vv'}^{i,a}}}{P_N + \sum_{i' \in \mathcal{V}, i' \neq i} \frac{P_{vv'}^{i',a} G_{i',a}^t G_{i',a}^r}{L_{vv'}^{i',a}}} \quad (9)$$

where $G_{i,a}^t$ and $G_{i,a}^r$ are the transmitting antenna gain at COV i and receiving antenna gain at RV a , respectively. We use a sectorial antenna model to approximate the actual

antenna pattern and main lobes of transmitter and receiver are aligned. Then, the antenna gain can be calculated based on the reference [27].

If $I_{i,j} = 1$ (i.e., option 1), then COV's associated RSU r_j will execute the task and it allocates $N_{CP}^{i,j}$ CPRBs to compute $Task_i$. The total latency of $Task_i(t)$ depends on $z_s^i, z_d^i, N_{CM}^{i,j}, N_{CP}^{i,j}$, which are the task's size, required computing resources, CMRBs and CPRBs allocated by the RSU. The total offloading latency of option 1 can be given as

$$t_{rv}^i = t_{v2r}^i + t_{cp}^i = \frac{z_s^i}{R_{rv}^i} + \frac{z_d^i}{N_{CP}^{i,j} F_b} \quad (10)$$

where t_{v2r}^i and t_{cp}^i are the time for transmitting the task and computing the task, respectively. If $I_{i,j}^a = 1$ (i.e., option 2), then by 2-hops offloading, the neighbor RV's associated RSU r_j will execute the task and it allocates $N_{CP}^{i,j}$ CPRBs to compute $Task_i$. The total offloading latency of option 2 can be calculated by

$$\begin{aligned} t_{rv}^i &= t_{v2v}^i + t_{v2r}^i + t_{cp}^i \\ &= \frac{z_s^i}{R_{vv'}^i} + \frac{z_s^i}{R_{rv}^i} + \frac{z_d^i}{N_{CP}^{i,j} F_b} \end{aligned} \quad (11)$$

where t_{v2v}^i is the time for relaying $Task_i$ from COV v_i to the chosen neighbor RV v_a via the V2V communication. It should be noted that we ignore the transmitting latency of computing results. We assume that the RSUs do not need to send the computing results back to the COVs, as it is likely that some particular regions rather than the COVs are interest in the tasks results in some applications (e.g., traffic congestion prediction, criminal recognition/prediction from image/video surveillance).

D. Problem Formulation

In this paper, considering that the message of offloading request, the offloading decision, and the computing result are relatively small, we ignore their transmission time. In order to maximize the system's offloading utility with consideration of load balance and latency constraints, the optimal offloading option and the corresponding allocated communication resources and computing resources need to be carefully optimized.

The first part of the system utility depends on the offloading capacity of RSUs weighted by the load-balance factors of both DTT load and CT load. We define the offloading capacity as the sum of the size of all successfully offloaded and executed tasks.

$$O_{i,1} = \left(\sum_{a \in \mathcal{V}} y_o^i(I_{i,j} + I_{i,j}^a) z_s^i \right) (L_{CP} + L_{CM}) \quad (12)$$

where L_{CP} and L_{CM} are the system CT and DTT load-balance factors based on the Jain's fairness [6]:

$$L_{CP} = \frac{\left(\sum_{j \in \mathcal{R}} N_{CP,a}^j \right)^2}{|\mathcal{R}| \left(\sum_{j \in \mathcal{R}} \left(N_{CP,a}^j \right)^2 \right)} \quad (13)$$

$$L_{CM} = \frac{\left(\sum_{j \in \mathcal{R}} N_{CM,a}^j \right)^2}{|\mathcal{R}| \left(\sum_{j \in \mathcal{R}} \left(N_{CM,a}^j \right)^2 \right)} \quad (14)$$

where $|\cdot|$ represents the cardinality of the set. $L_{CP}, L_{CM} \in \left[\frac{1}{|\mathcal{R}|}, 1 \right]$, the larger the value, the fairer it is. Moreover, if $N_{CP,a}^1 = N_{CP,a}^2 = \dots = N_{CP,a}^j = N_{CP,a}^R$ in Eq. (13), $L_{CP} = 1$, which means the remaining available computing resources of all RSUs are equal, thereby the allocations of computing tasks and resources are the fairest. The above also applies to Eq. (14).

The second part of the system utility depends on the reduction of latency,

$$O_{i,2} = z_l^i - t_{rv}^i \quad (15)$$

Since different metrics have different units, we need to conduct normalization operations to put them into a single objective utility function. The normalized functions are given as

$$O_{i,1}^{nor} = \frac{O_{i,1} - O_{1,min}}{O_{1,max} - O_{1,min}} \quad (16)$$

$$O_{i,2}^{nor} = \frac{O_{i,2} - O_{2,min}}{O_{2,max} - O_{2,min}} \quad (17)$$

where $O_{1,max}, O_{1,min}, O_{2,max}$, and $O_{2,min}$ are the maximum value and minimum value of the offloading capacity and the reduction of the task latency, respectively. The system's offloading utility function is

$$U_i = O_{i,1}^{nor} + O_{i,2}^{nor} \quad (18)$$

Hence, the computation task offloading optimization problem can be formulated as

$$\begin{aligned} & \max_{I_{i,j}, I_{i,j}^a, N_{CM}^{i,j}, N_{CP}^{i,j}} \sum_{i \in TASK} U_i \\ & \text{s.t. C1: } \sum_{i \in TASK} \sum_{a \in \mathcal{V}} y_o^i(I_{i,j} + I_{i,j}^a) N_{CM}^{i,j} \leq N_{CM,a}^j, \\ & \quad \forall j \in \mathcal{R} \\ & \text{C2: } \sum_{i \in TASK} \sum_{a \in \mathcal{V}} y_o^i(I_{i,j} + I_{i,j}^a) N_{CP}^{i,j} \leq N_{CP,a}^j, \\ & \quad \forall j \in \mathcal{R} \\ & \text{C3: } \sum_{j \in \mathcal{R}} I_{i,j} \leq 1, \quad \forall i \in TASK \\ & \text{C4: } \sum_{a \in \mathcal{V}} I_{i,j}^a \leq 1, \quad \forall i \in TASK, j \in \mathcal{R} \\ & \text{C5: } \sum_{j \in \mathcal{R}} \sum_{a \in \mathcal{V}} I_{i,j} + I_{i,j}^a \leq 1, \quad \forall i \in TASK \\ & \text{C6: } I_{i,j}, I_{i,j}^a \in \{0, 1\} \quad \forall i \in TASK, j \in \mathcal{R} \\ & \text{C7: } t_{rv}^i \leq z_l^i, \quad \forall i \in TASK \\ & \text{C8: } t_{v2v}^i \leq T_{V2V}^{i,a}, \quad \forall i \in TASK \\ & \text{C9: } t_{v2r}^i \leq T_{V2R}^{i,j}, \quad \forall i \in TASK \end{aligned} \quad (19)$$

where $T_{V2V}^{i,a}$ and $T_{V2R}^{i,j}$ denote the stable communication duration of the V2V transmission link and the V2R transmission link, respectively. The stable communication

TABLE I
VARIABLES IN THE SYSTEM MODEL

Variable	Meaning	Variable	Meaning
W_o	System observation period	$T_{V2V}^{i,a}, T_{V2R}^{i,j}$	Stable communication duration of the V2V/V2R transmission link
N	The number of time slot	W_l	Total communication resource of each RSU
$\mathcal{G}$	City grid set	N_{CM}	Total communication resource blocks (CMRBs) of each RSU
g_n	n -th grid	W_b	Bandwidth of each CMRB
$\mathcal{R}$	RSU set	W_h	Bandwidth of V2V communications
r_j	r -th RSU	F	Total computing resource of each RSU
$\mathcal{V}$	Vehicle set	N_{CP}	Total computing resource blocks (CPRBs) of each RSU
v_i	i -th vehicle	F_b	Computing resource of each CPRB
v_a	a -th relay vehicle	$N_{CM,a}^j$	Available communication resources of RSU r_j
$\mathbf{V}_i^{info}$	Vehicle information of v_i	$N_{CP,a}^j$	Available computing resources of RSU r_j
y_s^i	Vehicle ID	$\mathbf{RSU}_j^{info}$	RSU information of r_j
y_o^i	Task offloading indicator	$\mathbf{RSU}^{info}$	All RSUs information
$y_p^i = (x_i, y_i)$	Vehicle coordinate	$R_{rv}^i, R_{vv'}^i$	Effective V2R/V2V data rate
$y_m^i = (s_i, \theta_i)$	Speed vector	$SNR_{rv}^{i,j}$	SNR of V2R link
y_r^i	Associated RSUs index	$SINR_{vv'}^{i,a}$	SINR of V2V link
$\mathbf{y}_n^i$	Neighbor vehicles IDs	$P_{rv}^{i,j}, P_{vv'}^{i,a}$	Transmission power of V2R/V2V link
$\mathcal{V}^{info}$	All vehicles information	$L_{rv}^{i,j}, L_{vv'}^{i,a}$	Path loss of V2R/V2V link
$\mathbf{Task}_i$	Task offloading request	t_{rv}^i	Offloading latency
z_l^i	Remaining task time	t_{v2r}^i	Task transmitting latency for option 1
z_s^i	Tasks size	t_{v2v}^i	Task transmitting latency for option 2
z_d^i	Task required CPU cycles	t_{cp}^i	Task computing latency
$\mathcal{TASK}$	Task model of all vehicles	$O_{i,1}$	Offloading capacity
$I_{i,j}$	Whether the $\mathbf{Task}_i$ of COV v_i is offloaded to RSU r_j	L_{CP}	Computing traffic load-balance factor
$I_{i,j}^a$	Whether the $\mathbf{Task}_i$ of COV v_i is offloaded to RSU r_j via RV v_a	L_{CM}	Communication traffic load-balance factor
U_i	Offloading utility of task i	$O_{i,2}$	Latency reduction

duration can be seen as the duration that the transmission link can last. Their values can be calculated by the current position and the velocity of COV v_i and RV v_a [21]. The inequalities (C1-C2) reflect the constraints on the available communication resources and computing resources. Inequalities (C3-C6) ensure that a task is offloaded to only one RSU and at most one RV will be chosen. Inequality (C7) reflects the total offloading latency should be less than the remaining time to satisfy the tasks latency constraint. Inequalities (C8-C9) reflect the latency of V2R/V2V communications should be within the stable communication duration of the V2R/V2V links [17]. These two constraints can also prevent the handover cases during task offloading. If the task offloading latency t_{v2v}^i or t_{v2r}^i exceed the stable communication duration, in other words, if handover happens when vehicles move outside of the communication range/coverage of other vehicle/RSU during task offloading, the offloading utility will be set to 0, the task can be seen as failed. The goal of offloading optimization problem is to maximize the accumulated utility in Eq. (19). Hence, it will avoid the above case.

The above problem is a Mixed Integer Non-Linear Program (MINLP) problem which is also an NP-hard problem and cannot be resolved directly. The utility of computational tasks is non-linear and can be influenced by many factors,

including random task arrivals with different task features, RSUs' communication and computation loads varying with the task offloading decisions, and resource allocation granularities. Meanwhile, due to the mobility of vehicles, the V2R and V2V stable communication durations are both time-variant. Therefore, the widely used heuristic method suffers from low searching efficiency and high complexity. In the next section, we transform the above sequential decision-making problem into an Markov decision process (MDP) and use a DRL-based algorithm to solve it. The DRL agent located in the CCC does not need the accurate system dynamics model and it gradually improves its offloading policy toward achieving the optimal long-term performance based on the feedback reward of previous decisions.

IV. DRL-BASED COMPUTING OFFLOADING ALGORITHM

In this section, we formulate the utility maximization problem as an MDP. We firstly define the state space, action space and reward function of the formulated MDP. To solve the formulated MDP, RL methods (e.g., Monte Carlo, SARSA, expected SARSA, and Q-learning) using tabular solutions to store value functions are not suitable for our problem with high-dimensional state and action space considering the computational and storage consumptions. We prefer to use

DRL algorithm (i.e., Deep Q-Network) to find the optimal offloading policy of the problem and to avoid the curse of dimensionality [4], [8], [21], [29]. DQN uses a deep neural network (DNN) to estimate the action-value functions without keeping and updating a Q-table of Q-learning. Finally, the detailed implementation of our DRL-based algorithm is presented.

State Space: The DRL consists of an agent and the environment interacting with each other. In this scenario, the CCC is considered as an agent and the offloading requests, the available communication and computing resources of RSUs, the statuses of vehicles are regarded as the environment. Hence, the state space $\mathbf{S}$ is

$$\mathbf{S} = [\mathcal{TASK}, \mathcal{RSU}, \mathcal{V}^{info}] \quad (20)$$

Then, for each computing task $\mathbf{Task}_i$, the observed system state $s_t \in \mathbf{S}$, $s_t = [\mathbf{Task}_i, \mathcal{RSU}, \mathcal{V}^{info}]$. It should be noted that the RSUs information can either be predicted by the communication latency and computing latency based on tasks information or updated by itself. If both ways fails, the RSU status of both available communication resources and computing resources ($N_{CM,a}^j, N_{CF,a}^j$) will be set with value 0, which can be seen as falling silent.

Action Space: By observing the state, the agent conducts an action to determine the offloading option and the communication and computing resources allocations. Hence, the action space A conducted by the agent is

$$A = \left[(I_{i,j})_{s \times n}, (I_{i,j}^a)_{s \times s}, (N_{CM}^{i,j})_{s \times n}, (N_{CP}^{i,j})_{s \times n} \right] \quad (21)$$

Then, for each observed state s_t , the agent chooses an action $a_t \in \mathbf{A}$, $a_t = \left[(I_{i,j})_{1 \times n}, (I_{i,j}^a)_{1 \times s}, N_{CM}^{i,j}, N_{CP}^{i,j} \right]$

Reward Function: In each time slot, the agent conducts action a_t by observing state s_t , and then gets an immediate reward, which can be represented as $R(s_t, a_t) = U_i$. Our goal is to maximize the system utility in the observation period W_o which has N time slots. Then, our goal is to maximize the long-term accumulated reward can be given as

$$R = \sum_{i \in N} U_i \quad (22)$$

A. DQN Algorithm

The goal of DQN is to find an optimal offloading policy $a_t = \pi(s_t)$ for the upcoming offloading requests, so as to maximize the long-term accumulated reward. The state value function $v(s_t)$ and action-value function $Q(s_t, a_t)$ (i.e., Q-value) are used to represent the long-term reward, which is a prediction of the expected, accumulative, discounted, future reward. Q-value measures how good it is to take a particular action for a given state. DQN uses a DNN with parameters θ to predict Q-values of all possible actions $Q(s_t, a_t | \theta)$ for the current system state s_t . In state s_t , actions are taken by ϵ -greedy policy with ϵ decreasing linearly. The ϵ -greedy policy means that a random action is taken by probability ϵ and an action with the largest Q-value $Q(s_t, a_t | \theta)$ is taken with probability $(1 - \epsilon)$ to maximize the rewards [29]. After executing the selected action, agent obtains a reward $R(s_t, a_t)$ from the environment and the environment state changes to

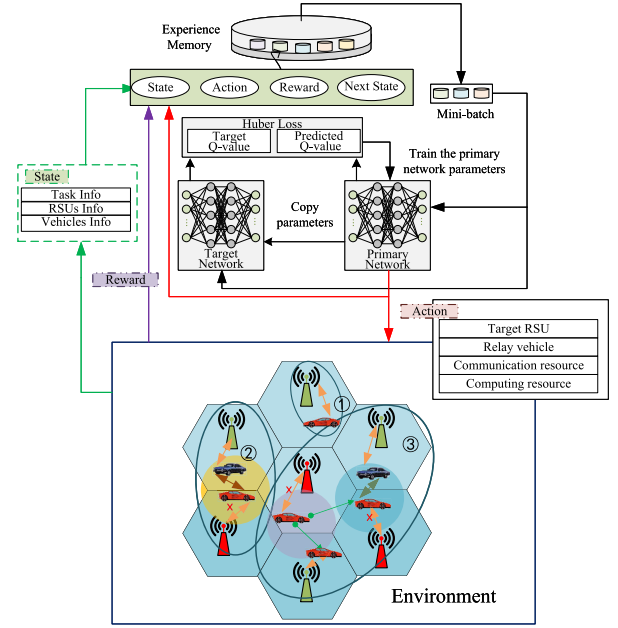


Fig. 2. An illustration of the deep Q-network framework.

a new state s'_t . DQN agent stores its interacting experience $(s_t, a_t, R(s_t, a_t), s'_t)$ at each time step in a replay memory. Then, a mini-batch of random samples $(s_t, a_t, R(s_t, a_t), s'_t)$ from the replay memory is used to train DNN. With the above experience replay, DQN can learn more from the past experience, and random selected mini-batches can prevent local minimum by decreasing the dependence of the collected experience. We use gradient descent approach to minimize the Huber Loss between the current predicted Q-value $Q(s_t, a_t | \theta)$ and the target Q-value $R(s_t, a_t) + \gamma \max_{a'_t} Q(s'_t, a'_t)$, during which the parameters θ of DNN are updated. γ is a discount factor reflecting the present value of future rewards. The Huber Loss function is defined as:

$$L_\delta(y', y) = \begin{cases} \frac{1}{2}(y' - y)^2 & \text{for } |y' - y| \leq \delta \\ \delta|y' - y| - \frac{1}{2}\delta^2 & \text{otherwise} \end{cases} \quad (23)$$

where y' is the target Q-value and y is the current predicted Q-value. The predicted Q-value directly approximates the optimal Q-value Q^* . Furthermore, to reduce the correlations between the predicted Q-value and the target Q-value and improve the stability and convergence, DQN uses a second neural network with independent parameters θ' to calculate the target Q-value, its parameters are only updated periodically (i.e., every T time steps) instead of every single time step. The modified target Q-value is $R(s_t, a_t) + \gamma \max_{a'_t} \hat{Q}(s'_t, a'_t | \theta')$. We give the pseudo code of our DRL-based computing offloading algorithm in Algorithm 1 and the DQN framework in Fig.2.

Algorithm 1 DRL-Based Computing Offloading Algorithm

- 1: INPUT: Initialize the experience replay buffer $\mathcal{M}$ of size S_M ; the mini-batch B of size $S_B < S_M$; a primary DQN and a target DQN with two sets of parameters θ and $\theta' = \theta$.
- 2: Initialize the replay memory $\mathcal{M}$ by random policy
- 3: // Interact with the environment


```

4: for all available episodes do
5:   for all time steps do
6:     The agent observes a state  $s_t$  and then picks an
       offloading action  $a_t(s_t)$  by the  $\epsilon$ -greedy policy;
7:     Execute the action  $a_t(s_t)$  to the environment to obtain
       the reward  $R(s_t, a_t)$  and observe the next state  $s'_t$ ;
8:     Store the experience tuple  $(s_t, a_t, R(s_t, a_t), s'_t)$  in the
       experience replay buffer  $M$  to update it;
9:     // Updates the DQNs' parameters
10:    Randomly sample mini-batch samples  $B$  from
        memory  $M$ , i.e., sampling  $(s_t, a_t, R(s_t, a_t), s'_t)$  from
        memory  $M$ ;
11:    for all  $B$  samples  $(s_t, a_t, R(s_t, a_t), s'_t)$  in the mini-
        batch transitions do
12:      if the current episode ends at time step  $i + 1$  then
13:        set the target Q-value  $y' = R(s_t, a_t)$ 
14:      else
15:        set  $y' = R(s_t, a_t) + \gamma \max_{a'_t} \hat{Q}(s'_t, a'_t | \theta')$ ;
16:      end if
17:      Conduct gradient descent with Huber Loss to
        update parameters  $\theta$  of primary network;
18:    end for
19:    The agent regularly reset  $\theta'(i + 1) = \theta(i)$  every  $T$ 
        time steps, and otherwise  $\theta'(i + 1) = \theta'(i)$ ;
20:  end for
21: end for
22: OUTPUT: All offloading actions.

```

DQN chooses actions according to the random policy to initialize the training samples in the experience replay buffer and chooses actions according to the ϵ -greedy strategy in the stage of Q network training. Then, new experience tuples are generated by interactions with the environment and updated the old ones in the experience buffer. These experience tuples participate in the Q network training through random small-batch sampling, which destroys the strong correlation between the training samples, thereby improving the training effectiveness. The above process is repeated for training and policy optimization until the algorithm stop condition is met. It should be noted that the immediate reward is set to be 0 if the action does not satisfy the constraints in optimization problem (19), and the computing offloading task can be seen as failed.

B. Complexity Analysis

In our proposed algorithm, the Q-networks are fully-connected networks that contain two hidden layers. We denote the hidden layers of target Q-network as L_1, L_2 , the dimension of state space is $D_{state} = 3 + 2 \times n + (7 + y_n) \times s$, and the dimension of action space is $D_{action} = 1 \times n + 1 \times s + 2 = n + s + 2$. Then, the complexity of Q-value generation is

$$O((s(7 + y_n) + 2n + 3)L_1 + L_1L_2 + L_2(n + s + 2)) \quad (24)$$

Since the size of hidden layers and y_n is fixed, and the number of RSUs is usually less than that of vehicles, the complexity of Q-value generation can be given as $O(s)$. In the training process of the Q-networks, we assume there are N time slots in a period, then the Q-networks will be trained in N_1 time steps. In each time step, the complexity is similar to the Q-value

TABLE II
SIMULATION PARAMETERS

Symbol	Value
The number of grids and RSUs $ \mathcal{G} , \mathcal{R} , n$	19
The number of COVs	100
The number of RVs	35
The number of vehicles $ \mathcal{V} , s$	135
The number of neighbor vehicles y_N	[0,3]
Vehicle velocity s_i	[70,120]
Driving direction θ_i	$[0, \pi/2, \pi, 3\pi/2]$
Latency constraint z_r^i , (ms)	{25,100}
Data size of task z_s^i , (MB)	[80,800]
Required computation resources of task z_d^i , (Mcycles)	[0.6,6]
RSUs computational capability F , (GHz)	2
V2V bandwidth W_h , (MHz)	20
SNR and SINR for V2R and V2V, (dB)	[-10,30]
Size of communication resource block (CMRB) W_b , (MHz)	1
The number of CMRBs N_{CM}	100
Size of computing resource block (CPRB) F_b , (MHz)	10
The number of CPRBs N_{CP}	200
Allocation granularity of communication resource (CMRB)	[5,10,15,20]
Allocation granularity of computing resource (CPRB)	[10,15,20,25,30]
Parameters for DNN	
Huber loss, δ	1
Learning rate, α_{lr}	0.00025
Memory size, S_M	5000
Batch size, S_B	128
Discount factor, γ	0.99
ϵ_{max}	1
ϵ_{min}	0.01
Decay speed of epsilon, λ	0.001
Target NN's parameters updating period, T	100 time steps

predictor. Finally, in an observation period, the complexity of our proposed algorithm is $O(N \times s)$.

V. SIMULATIONS AND PERFORMANCE ANALYSIS

The simulations are conducted on a desktop which has a 4G RAM and an Intel(R) Core(TM) i7-4720HQ CPU 2.60GHZ. The simulation platform is based on Spider with Python 3.7 on Windows 10. In our proposed algorithm, the action-value function is approximated by DNNs. The parameters used in our simulation experiments are summarized in Table II. It should be noted that this paper does not consider interference management or the energy efficiency optimization, we do not optimize the transmission power of the communication resource. Hence, the random distributions of SNR and SINR, which are determined by the transmitting power, large-scale fading, small-scale fading, noise, and interference, are given for simulation analysis. When the above values are known by measurements and calculations, the SNR/SINR is obtained, thereby the proposed optimization model and optimization algorithm are still valid. This is mainly because the solution algorithm of the proposed optimization model is based on the DRL algorithm rather than the mathematical programming method. In the mathematical programming method, the mapping relationship between SNR, SINR and communication rate of the logarithmic function may affect mathematical conversions. However, the DRL algorithm does not involve any mathematical conversion. It only takes SNR or SINR values and uses Shannon's formula to calculate the

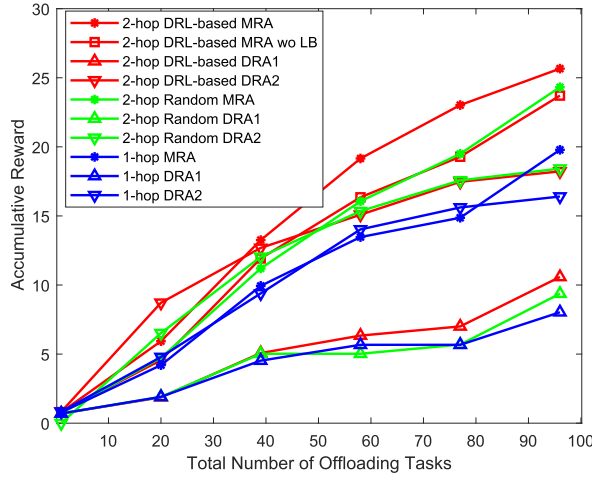


Fig. 3. Accumulative reward vs. Number of tasks.

communication rate and the reward function, which will not affect the effectiveness of the communication bandwidth and transmission hops optimization in this paper.

To the best of our knowledge, there is no work jointly involving the multi-resources allocation, nodes selection, traffic load-balancing, and optimal matching of computing resources allocation and communicating resources allocation with consideration of different latency requirements in vehicular edge computing scenario. The system scenario, the involved multi-dimensional network resources, and the optimized performance make our optimization model including the optimization objective and constraints, the corresponding definitions for the state, action and reward of DRL algorithm very different with existing works. Hence, it is difficult to make our paper comparable directly with existing works. In addition, since the performance improvement of our work is the result of the offloading optimization policy rather than the improvement of DRL algorithm itself, it is unnecessary to compare our method with other DRL algorithms. Therefore, in this paper, to demonstrate the effectiveness of our proposed algorithm, we introduce the following state-of-the-art algorithms for the purpose of comparison: 1) 2-hop Random Offloading Algorithm; 2) 1-hop Local Offloading Algorithm; 3) 2-hop DRL-based Offloading Algorithm without Load-balancing. In each algorithm, there are DRA standing for determined-grained communication and computing resources allocations and MRA standing for multi-grained optional communication and computing resources allocations. Particularly, we set the DRA1 to be determined-grained resource allocation with 5 CMRBs and 10 CPRBs, and DRA2 with 20 CMRBs and 30 CPRBs.

- 1) 2-hop Random Offloading Algorithm (2-hop Random MRA, 2-hop Random DRA1, 2-hop Random DRA2): Task offloading options include 2-hop VAO and the offloading option selections and the communication and computing resources allocations are random.
- 2) 1-hop Local Offloading Algorithm (1-hop MRA, 1-hop DRA1, 1-hop DRA2): Task offloading options do not include 2-hop VAO, all the tasks are offloaded to the COVs' associated RSUs via 1-hop V2R communications.

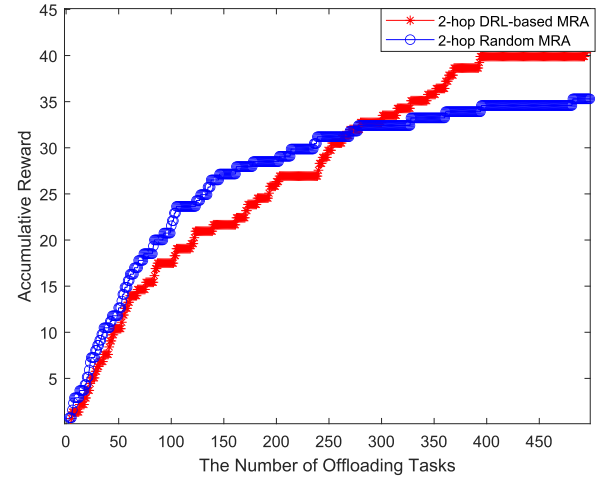


Fig. 4. Accumulative reward vs. Number of tasks.

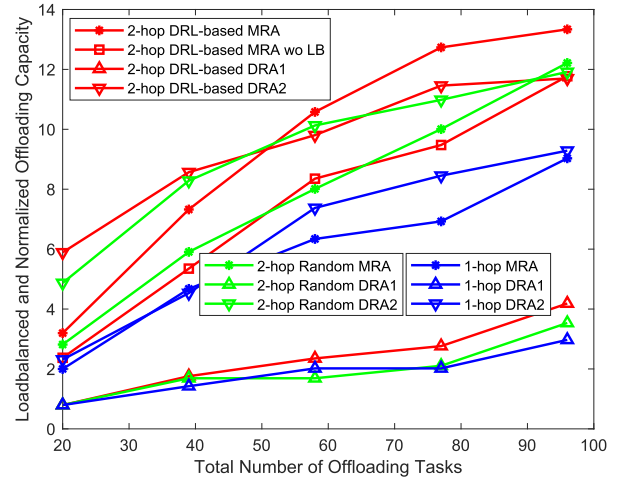


Fig. 5. Accumulative weighted offloading capacity vs. Number of tasks.

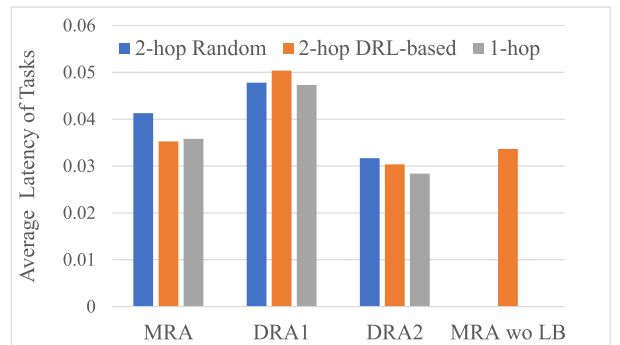


Fig. 6. Average latency of successfully offloaded tasks (in s).

- 3) 2-hop DRL-based Offloading Algorithm without Load-balancing (2-hop DRL-based MRA w.o. LB): Task offloading options include 2-hop VAO. Only the offloading capacity of the system utility is not weighted by the load-balance factor, others are same with the proposed DRL-based algorithm, in which the offloading options and communication and computing resources allocations are optimized.

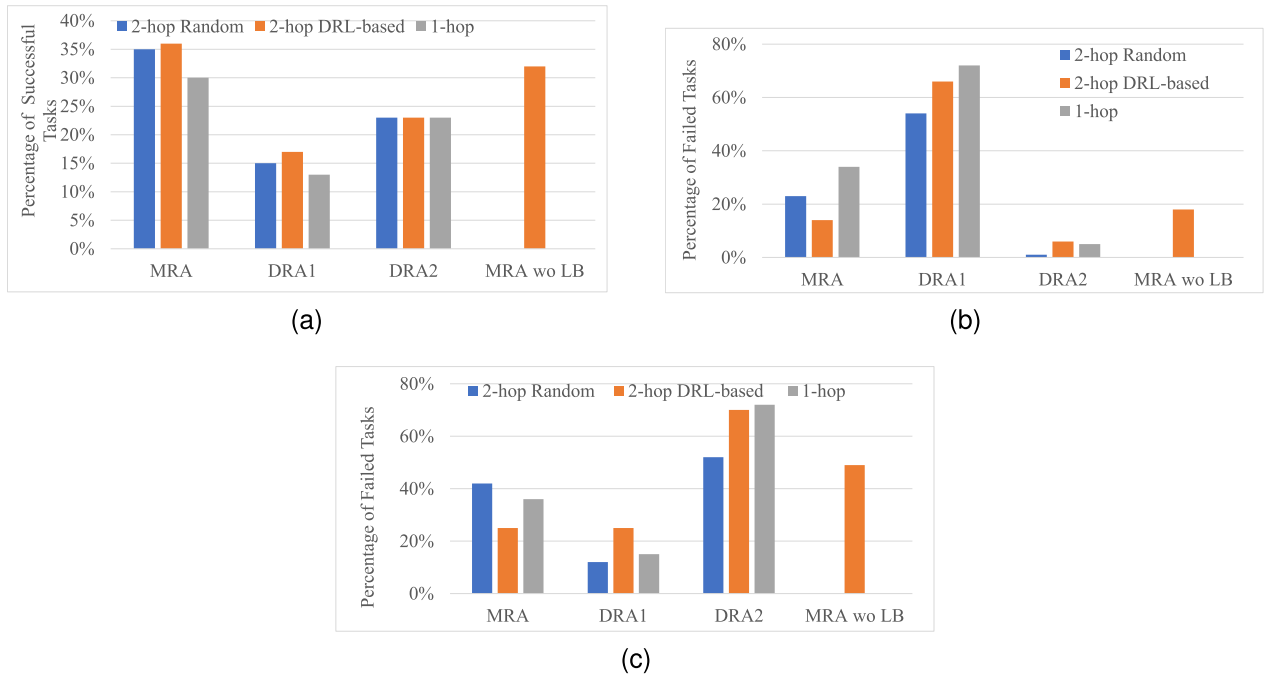


Fig. 7. The percentage of successful and failed tasks. (a) The percentage of successfully offloading tasks, (b) The percentage of failed offloading tasks due to stable communication duration or latency constraint are exceeded, (c) The percentage of failed offloading tasks due to insufficient communication or computing resources.

A. Accumulative Reward

In Fig. 3, we evaluate the long-term accumulative reward of the proposed 2-hop DRL-based MRA algorithm over other algorithms. We can see that the 2-hop DRL-based algorithms outperform 2-hop Random and 1-hop algorithms in both determined-grained and multi-grained optional resource allocations. It is because the 2-hop Random and 1-hop algorithms do not consider the load-balancing and optimal offloading options and resources allocations. 1-hop algorithms have the worst performance because it only selects the associated RSU to offload, whereas the spatial-temporal traffic tide phenomenon results nonuniform traffic loads on both communication and computation resources. Hence, some RSUs might be overloaded and congested while other RSUs are underloaded, thereby the resource utilization efficiency is low. Similarly, 2-hop Random algorithms randomly make offloading decisions of the offloading option and resources allocations, these offloading decisions may mismatch with the tasks' features and RSUs' available resources. 2-hop DRL-based MRA w.o. LB, 2-hop DRL-based DRA algorithms obtain smaller reward than the proposed 2-hop DRL-based MRA algorithm. Although 2-hop DRL-based MRA w.o. LB makes the optimal offloading options and communication and computing resources allocations, it ignores the load-balance influences. Hence, it might decide to offload many tasks to the same RSU via 2-hops VOA, resulting this RSU overloaded and missing other more appropriate tasks. All the algorithms with MRA have higher reward than those with DRA for the reason that they have more flexible resources allocations according to various computing task features. We further give the convergence curves of accumulative rewards with increasing numbers of offloading tasks in Fig. 4 comparing the proposed 2-hop DRL-based MRA algorithm and 2-hop Random MRA algorithms. As the number of

offloading tasks increases, the increasing accumulative reward slows down and gradually converges to a constant. This is because there is no available RVs or available RSUs communication and computing resources for these burst concurrent offloading tasks in an observation period.

B. Improvement of Load-Balanced Offloading Capacity

In Fig. 5, we further evaluate the system accumulated offloading capacity weighted by the load-balance factor, we apply normalization on the weighted offloading capacity for better readability. Since the proposed 2-hop DRL-based MRA algorithm jointly considers the optimal offloading options, resources allocations, and load balance according to tasks' features, vehicles' mobility, and RSUs' loads, it has the best performance among all the 2-hop DRL-based algorithms. All the algorithms with DRA1 have the worst performance because that they allocate the fewest communication and computing resources (i.e., fewest CMRBs and CPRBs). Although, fewer resources give more chances for tasks to be offloaded, the transmitting latency (i.e., t_{v2v}^i and t_{v2r}^i) may exceed the stable communication duration of V2V/V2R transmission links (i.e., $T_{V2V}^{i,a}$ and $T_{V2R}^{i,j}$) due to the relative mobilities among vehicles or between vehicles and RSUs. Also, the total latency t_{rv}^i may exceed the latency constraint. The above reasons result in increasing failed tasks. Compared with algorithms with DRA1, algorithms with DRA2 allocate more CMRBs and CPRBs to tasks, thereby relief the task failures due to mobility and latency constraints at the expense of less task offloading chances. When the tasks are featured by large data size, computation-intensive, low latency-constrained, and requested by high-mobility vehicles, the algorithms with DRA2 will match better with the task features than the algorithms with DRA1, as shown in Fig. 5.

Although the 2-hop Random algorithms do not perform any optimizations, it has more various offloading options than the 1-hop algorithms, thereby achieving larger offloading capacities.

C. Average Latency of Tasks

We evaluate the average latency of those successful offloading tasks in Fig. 6. The performance results reveal that the algorithms with DRA1 have longer latency than others due to allocations of fewer CMRBs and CPRBs. As a result, they have more task failures when there are more tasks coming with the features of high mobility and strict latency constraints. Whereas the algorithms of DRA2 have the shortest latency and algorithms with MRA have modest performance for its flexible resource allocation granularities. It is worth noting that although the 1-hop algorithms have relative short latency mainly because that they only have 1-hop communication delay, their offloading capacities and the accumulative rewards are the worst. It should be noted that the latency performance of our proposed 2-hop DRL-based with MRA is very competitive.

Fig. 7 is used to further analyze the underlying reasons behind the above simulation results. We can see that the algorithms with DRA1 have fewer tasks that are offloaded successfully than the algorithms with DRA2 as shown in Fig. 7(a). They allocate fewer CMRBs and CPRBs, then they have more failed tasks due to the stable communication duration or latency constraint are exceeded. The algorithms with DRA2 are the opposite, they allocate more resources to tasks therefore the insufficient communication or computing resources lead to fewer offloading chances. According to the task features of high mobility and severer latency constraints, the reason of exceeding stable link duration or latency constraint plays a dominant role. It is worth noting that our proposed 2-hop DRL-based algorithm with MRA has the highest percentage of successful offloading tasks compared with all other algorithms, and from Figs. 7(b) and 7(c) we can see that the two failed reasons account for a more similar proportion compared with other algorithms. The 2-hop DRL-based algorithm MRA w.o. LB does not consider the load balance in its optimization, the mismatch of RSUs' load distributions, resources allocations, and task features makes it perform worse than the proposed 2-hop DRL-based MRA algorithm.

VI. CONCLUSION

We propose an intelligent load-balanced vehicular edge computing scheme in which a two-hop vehicle-assisted computation offloading is explored. The task offloading optimization problem is formulated and transferred into an MDP aiming at maximizing the offloading capacity while increasing both the number of offloading tasks and latency reduction. Based on the DRL algorithm, we solve the optimization problem and compare our scheme with other state-of-the-art algorithms in the simulation experiments. From the simulation results and analysis, on one hand, edge computation offloading without considering flexible and optimal resources allocations of both communication aspect and computing aspect will result in failed task offloading for

the reason of task transmitting time or total latency exceeding stable communication duration or latency constraint, and insufficient communication/computing resources. On the other hand, edge computation offloading without considering load balance will result in congestions on overloaded RSUs and under-utilization of underloaded RSUs' resources. Our proposed 2-hop DRL-based algorithm with MRA deals with the above problems by optimally matching the RSUs' features of computing and communication loads, resources allocations, and tasks' features of mobility and latency constraints, thereby achieving the best task offloading utility in both offloading capacity and latency-reduction.

REFERENCES

- [1] W. Liu, Y. Watanabe, and Y. Shoji, "Vehicle-assisted data delivery in smart city: A deep learning approach," *IEEE Trans. Veh. Technol.*, vol. 69, no. 11, pp. 13849–13860, Nov. 2020.
- [2] C. Yang, W. Lou, Y. Liu, and S. Xie, "Resource allocation for edge computing-based vehicle platoon on freeway: A contract-optimization approach," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 15988–16000, Dec. 2020.
- [3] Q. Wang, L. T. Tan, R. Q. Hu, and Y. Qian, "Hierarchical energy-efficient mobile-edge computing in IoT networks," *IEEE Internet Things J.*, vol. 7, no. 12, pp. 11626–11639, Dec. 2020.
- [4] Y. Liu, H. Yu, S. Xie, and Y. Zhang, "Deep reinforcement learning for offloading and resource allocation in vehicle edge computing and networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 11, pp. 11158–11168, Nov. 2019.
- [5] X. Gong and Y. Lu, "Data mining based research on urban tide traffic problem," in *Proc. 11th Int. IEEE Conf. Intell. Transp. Syst.*, Oct. 2008, pp. 122–127.
- [6] B. Fan, Z. He, Y. Wu, J. He, Y. Chen, and L. Jiang, "Deep learning empowered traffic offloading in intelligent software defined cellular V2X networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 11, pp. 13328–13340, Nov. 2020.
- [7] Y. Dai, D. Xu, S. Maharjan, and Y. Zhang, "Joint load balancing and offloading in vehicular edge computing and networks," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4377–4387, Jun. 2019.
- [8] H. Lu, X. He, M. Du, X. Ruan, Y. Sun, and K. Wang, "Edge QoE: Computation offloading with deep reinforcement learning for Internet of Things," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9255–9265, Oct. 2020.
- [9] G. Qiao, S. Leng, K. Zhang, and Y. He, "Collaborative task offloading in vehicular edge multi-access networks," *IEEE Commun. Mag.*, vol. 56, no. 8, pp. 48–54, Aug. 2018.
- [10] M. Li, P. Si, and Y. Zhang, "Delay-tolerant data traffic to software-defined vehicular networks with mobile edge computing in smart city," *IEEE Trans. Veh. Technol.*, vol. 67, no. 10, pp. 9073–9086, Oct. 2018.
- [11] H. Peng and X. Shen, "Multi-agent reinforcement learning based resource management in MEC- and UAV-assisted vehicular networks," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 1, pp. 131–141, Jan. 2021.
- [12] J. Zhou, D. Tian, Z. Sheng, X. Duan, and X. Shen, "Joint mobility, communication and computation optimization for UAVs in air-ground cooperative networks," *IEEE Trans. Veh. Technol.*, vol. 70, no. 3, pp. 2493–2507, Mar. 2021.
- [13] X. Hou et al., "Reliable computation offloading for edge-computing-enabled software-defined IoT," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 7097–7111, Aug. 2020.
- [14] K. Wang, H. Yin, W. Quan, and G. Min, "Enabling collaborative edge computing for software defined vehicular networks," *IEEE Netw.*, vol. 32, no. 5, pp. 112–117, Sep. 2018.
- [15] Z. Mlika and S. Cherkaoui, "Network slicing with MEC and deep reinforcement learning for the Internet of Vehicles," *IEEE Netw.*, vol. 35, no. 3, pp. 132–138, Jun. 2021.
- [16] J. Tang, Y. Duan, Y. Zhou, and J. Jin, "Distributed slice selection-based computation offloading for intelligent vehicular networks," *IEEE Open J. Veh. Technol.*, vol. 2, pp. 261–271, 2021.
- [17] Q. Zhang, L. Gui, F. Hou, J. Chen, S. Zhu, and F. Tian, "Dynamic task offloading and resource allocation for mobile-edge computing in dense cloud RAN," *IEEE Internet Things J.*, vol. 7, no. 4, pp. 3282–3299, Apr. 2020.

- [18] L. Feng, W. Li, Y. Lin, L. Zhu, S. Guo, and Z. Zhen, "Joint computation offloading and URLLC resource allocation for collaborative MEC assisted cellular-V2X networks," *IEEE Access*, vol. 8, pp. 24914–24926, 2020.
- [19] Y. Wang et al., "A game-based computation offloading method in vehicular multiaccess edge computing networks," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 4987–4996, Jun. 2020.
- [20] Z. Ning et al., "Partial computation offloading and adaptive task scheduling for 5G-enabled vehicular networks," *IEEE Trans. Mobile Comput.*, vol. 21, no. 4, pp. 1319–1333, Apr. 2022.
- [21] L. T. Tan and R. Q. Hu, "Mobility-aware edge caching and computing in vehicle networks: A deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 67, no. 11, pp. 10190–10203, Nov. 2018.
- [22] K. Zhang, Y. Zhu, S. Leng, Y. He, S. Maharjan, and Y. Zhang, "Deep learning empowered task offloading for mobile edge computing in urban informatics," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 7635–7647, Oct. 2019.
- [23] X. Wang, Y. Han, C. Wang, Q. Zhao, X. Chen, and M. Chen, "In-edge AI: Intelligentizing mobile edge computing, caching and communication by federated learning," *IEEE Netw.*, vol. 33, no. 5, pp. 156–165, Sep. 2019.
- [24] W. Zhan et al., "Deep-reinforcement-learning-based offloading scheduling for vehicular edge computing," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 5449–5465, Jun. 2020.
- [25] Y. Zhou, L. Tian, L. Liu, and Y. Qi, "Fog computing enabled future mobile communication networks: A convergence of communication and computing," *IEEE Commun. Mag.*, vol. 57, no. 5, pp. 20–27, May 2019.
- [26] J. Shi, J. Du, J. Wang, J. Wang, and J. Yuan, "Priority-aware task offloading in vehicular fog computing based on deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 16067–16081, Dec. 2020.
- [27] Y. Wu, L. Yan, and X. Fang, "A low-latency content dissemination scheme for mmWave vehicular networks," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 7921–7933, Oct. 2019.
- [28] G. Sun, H. Al-Ward, G. O. Boateng, and G. Liu, "Autonomous cache resource slicing and content placement at virtualized mobile edge network," *IEEE Access*, vol. 7, pp. 84727–84743, 2019.
- [29] R. Li et al., "Deep reinforcement learning for resource management in network slicing," *IEEE Access*, vol. 6, pp. 74429–74441, Nov. 2018.
- [30] L. Liu, C. Chen, Q. Pei, S. Maharjan, and Y. Zhang, "Vehicular edge computing and networking: A survey," *Mobile Netw. Appl.*, vol. 26, no. 3, pp. 1145–1168, Jun. 2021.
- [31] N. Cheng et al., "Space/aerial-assisted computing offloading for IoT applications: A learning-based approach," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 5, pp. 1117–1129, May 2019.
- [32] G. Qiao, S. Leng, S. Maharjan, Y. Zhang, and N. Ansari, "Deep reinforcement learning for cooperative content caching in vehicular edge computing and networks," *IEEE Internet Things J.*, vol. 7, no. 1, pp. 247–257, Jan. 2020.
- [33] C. Zhou et al., "Deep reinforcement learning for delay-oriented IoT task scheduling in SAGIN," *IEEE Trans. Wireless Commun.*, vol. 20, no. 2, pp. 911–925, Feb. 2021.
- [34] 3GPP. (Oct. 31, 2023). *3GPP Study on Evaluation Methodology of New Vehicle-to-Everything (V2X) Use Cases for LTE and NR (v15.3.0, Release 15)*. [Online]. Available: <https://portal.3gpp.org/Specifications.aspx?q=1&series=31&releases=all&draft=False&underCC=False&withACC=False&withBCC=False&numberNYA=False>



Yu Wu received the B.E. degree and the Ph.D. degree in communication engineering from Southwest Jiaotong University, Chengdu, China, in 2014 and 2023, respectively. Her research interests include vehicular network, mobile edge computing, resource optimization, PAPR reduction, and deep reinforcement learning.



Xuming Fang (Senior Member, IEEE) received the B.E. degree in electrical engineering, the M.E. degree in computer engineering, and the Ph.D. degree in communication engineering from Southwest Jiaotong University, Chengdu, China, in 1984, 1989, and 1999, respectively. He held visiting positions with Technical University Berlin, Berlin, Germany, and also with the University of Texas at Dallas, Richardson, TX, USA. He is currently a Professor with the School of Information Science and Technology, Southwest Jiaotong University, China. He has published over 200 high-quality research papers in journals and conference publications. His research interests include radio resource management, AI/ML learning for wireless communication networks, integration of communication, sensing, and computing, and intelligent wireless access for high-speed railway. He was the Chair of IEEE Vehicular Technology Society of Chengdu Chapter and has been an Editor of several journals, including IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY.



Geyong Min (Member, IEEE) received the B.Sc. degree in computer science from the Huazhong University of Science and Technology, China, in 1995, and the Ph.D. degree in computing science from the University of Glasgow, U.K., in 2003. He is currently a Professor of high-performance computing and networking with the Department of Computer Science, College of Engineering, Mathematics and Physical Sciences, University of Exeter, U.K. His research interests include computer networks, wireless communications, parallel and distributed computing, ubiquitous computing, multimedia systems, modeling, and performance engineering.



Hongyang Chen (Senior Member, IEEE) received the B.S. and M.S. degrees from Southwest Jiaotong University, Chengdu, China, in 2003 and 2006, respectively, and the Ph.D. degree from The University of Tokyo, Tokyo, Japan, in 2011. From 2011 to 2020, he was a Researcher with Fujitsu Ltd., Tokyo. He is currently a Senior Research Expert with the Zhejiang Laboratory, China. He is an Adjunct Professor with Zhejiang University. He has authored or co-authored more than 150 refereed journal and conference papers in IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON SIGNAL PROCESSING, IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, IEEE TRANSACTIONS ON COMMUNICATIONS, SIGMOD, KDD, IJCAI, AAAI, and NeurIPS, and has been granted more than 30 PCT patents. His research interests include data-driven intelligent systems, graph machine learning, big data mining, and LLM. He was the symposium chair or the special session organizer of some flagship conferences. He was an editor of IEEE journals. He was the Leading Guest Editor of IEEE JOURNAL ON SELECTED TOPICS OF SIGNAL PROCESSING on tensor decomposition. He has been selected as the Distinguished Lecturer of the IEEE Communication Society from 2021 to 2023.



Chunbo Luo (Senior Member, IEEE) received the Ph.D. degree in high-performance cooperative wireless networks from the University of Reading, Reading, U.K., in 2011. His research has been supported by UKRI, Royal Society, EU H2020, and industries. His research interest focuses on developing model-based and machine-learning algorithms to solve networking and engineering problems, with a particular focus on networked unmanned aerial vehicle. He is a fellow of the Higher Education Academy.